

## 6.1 Historical Perspective

### Early Generations

**Machine language** is a programming language consisting of numeric instructions that a computer can execute directly.

**Debugging** is the process of locating and correcting errors in a program.

**Assembly language(汇编语言)** is a mnemonic representation of machine instructions that uses symbolic names instead of numeric codes.

**Mnemonics(助记符)** Mnemonics are symbolic abbreviations 缩写 (such as MOV, ADD) used to represent machine instructions.

**Identifiers(标识符)** are descriptive names used to represent variables or memory locations in programs.

**Assemblers(汇编器)** are programs that translate assembly language into machine language.

**Second-generation languages(第二代语言)** Assembly languages are called second-generation languages.

**Machine-dependent** language relies on the specific architecture of a computer. (不可移植)

**machine independent** language does not depend on a specific computer architecture. (可移植性强)

-**Dialects** are variations of the same programming language caused by differences in implementation on different machines.

### Third Generation Languages (第三代语言)

**High-level primitives(原语)** are programming constructs that express operations in larger, more abstract steps.

**Third-generation languages** use high-level primitives and are machine independent.

### Translation Mechanism (翻译机制)

-**translator** 翻译

A translator is a program that converts high-level language code into machine language.

-**compilers** 编译

A compiler translates an entire program into machine language before execution.

-**interpreters** 解释

An interpreter translates and executes instructions line by line.

### Language Types (语言类型)

-**natural languages**

Natural languages are human languages that evolved over time, such as English and German.

-**formal languages**

Formal languages are precisely defined by strict grammatical rules, such as programming languages.

高级语言 → (compiler) → 汇编语言 → (assembler) → 机器语言

### Machine Independence and Beyond

Machine independence allows programs to run on different machines, but in practice, differences in compilers and I/O systems lead to dialects.

Standards such as ANSI and ISO help reduce these differences, but language extensions may still cause compatibility issues.

### Cross-Platform Software

Cross-platform software goes further by being independent of both hardware and operating systems, allowing programs to run across networks.

**Programming Paradigms (编程范式)**

**programming paradigms** are different approaches to solving problems and developing software.

Figure 6.1 Generations of programming languages

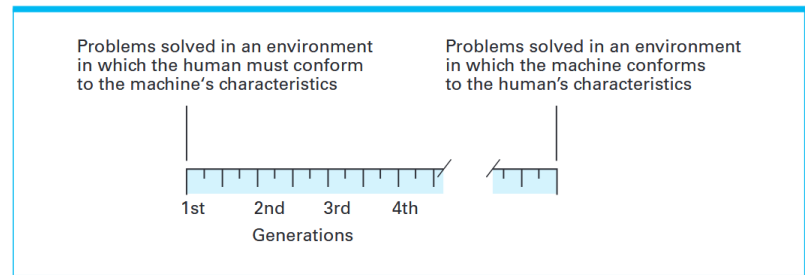
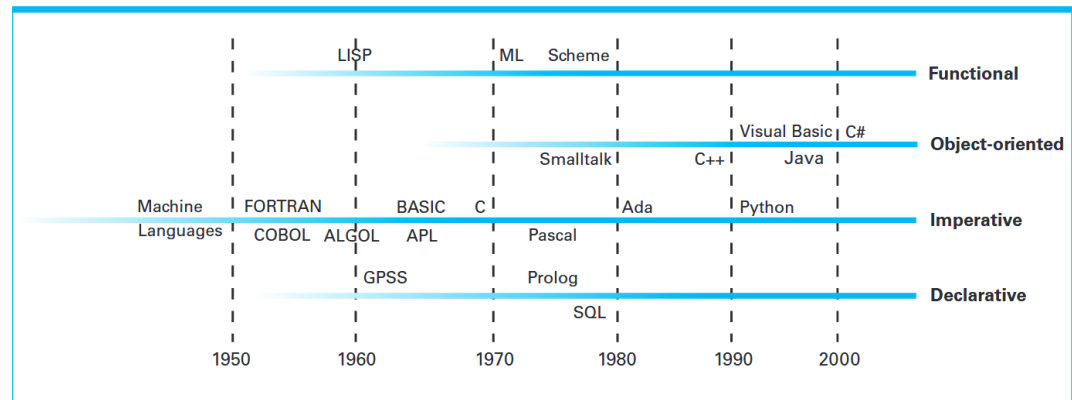


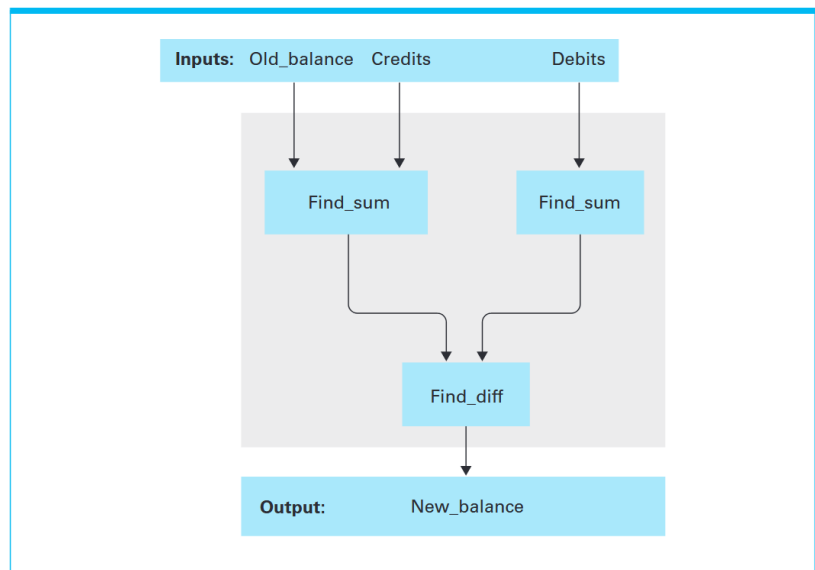
Figure 6.2 The evolution of programming paradigms



**imperative paradigm (procedural paradigm) (命令式范式)**

The imperative paradigm defines programming as writing a sequence of commands that manipulate data.命令式范式将编程定义为编写一系列操作数据的指令。

Figure 6.3 A function for checkbook balancing constructed from simpler functions



**declarative paradigm (声明式范式)**

The declarative paradigm focuses on describing what problem to solve rather than how to solve it.声明式范式关注“做什么”，而不是“怎么做”。

### -logic programming (逻辑编程)

Logic programming is a form of declarative programming based on formal logic.

逻辑编程是一种基于形式逻辑的声明式编程。

### functional paradigm (函数式范式)

The functional paradigm views a program as a function that maps inputs to outputs.

函数式范式将程序视为输入到输出的映射函数。

(Find\_diff

(Find\_sum Old\_balance Credits)

(Find\_sum Debits)

) 函数式范式通过函数组合完成计算，结果直接传递，不存中间变量。

### object-oriented paradigm (面向对象范式)

The object-oriented paradigm models a system as a collection of interacting objects.

### -object-oriented programming (OOP)

Object-oriented programming (OOP) is the programming process based on the object-oriented paradigm.

### -objects

Objects are units that contain both data and the ability to perform actions.

### -methods

Methods are functions associated with an object that define its behavior.

```
s1 = Student("Alice", 90)
s2 = Student("Bob", 85)
```

### -class

A class is a description or template that defines the properties and methods of objects.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def introduce(self):
        print("My name is", self.name)

    def study(self):
        print(self.name, "is studying")
```

### -instance

An instance is an object created from a class.

```
s1.introduce()
s1.study()
```

| Paradigm                   | 核心问题       | 核心思想       | 关键词                            | 一句话理解       |
|----------------------------|------------|------------|--------------------------------|-------------|
| Imperative<br>(Procedural) | How (怎么做)  | 按步骤执行指令    | algorithm, sequence, variables | 一步一步算，存中间结果 |
| Declarative                | What (做什么) | 描述问题，让系统求解 | description, logic, rules      | 不写步骤，只说目标   |

| Paradigm              | 核心问题            | 核心思想      | 关键词                             | 一句话理解       |
|-----------------------|-----------------|-----------|---------------------------------|-------------|
| Functional            | Function (函数组合) | 用函数组合得到结果 | function, composition, no state | 函数嵌套, 直接出结果 |
| Object-Oriented (OOP) | Objects (对象)    | 用对象协作解决问题 | object, class, method           | 对象之间互相调用    |

**1. In what sense is a program in a third-generation language machine independent? In what sense is it still machine dependent?**

Its statements do not refer to the characteristics of any particular machine and can be compiled for different machines.

However, it is still machine dependent in that compilers may reflect characteristics of the underlying machine, and differences such as I/O handling may require program modification.

**2. What is the difference between an assembler and a compiler?**

高级语言 → (compiler) → 汇编语言 → (assembler) → 机器语言

An assembler translates assembly language into machine language, whereas a compiler translates high-level language programs into machine language.

高级语言 → translator → 机器语言

**3. We can summarize the imperative programming paradigm by saying that it places emphasis on describing a process that leads to the solution of the problem at hand. Give a similar summary of the declarative, functional, and object-oriented paradigms.**

**①imperative paradigm**

describing a process that leads to the solution of the problem at hand

**②declarative paradigm**

describing what problem to solve rather than how to solve it.

**③functional paradigm**

viewing a program as a function that maps inputs to outputs.

**④object-oriented paradigm**

modeling a system as a collection of interacting objects.

**4. In what sense are the third-generation programming languages at a higher level than the earlier generations?**

Third-generation languages use high-level primitives and are machine independent.

## 6.2 Traditional Programming Concepts

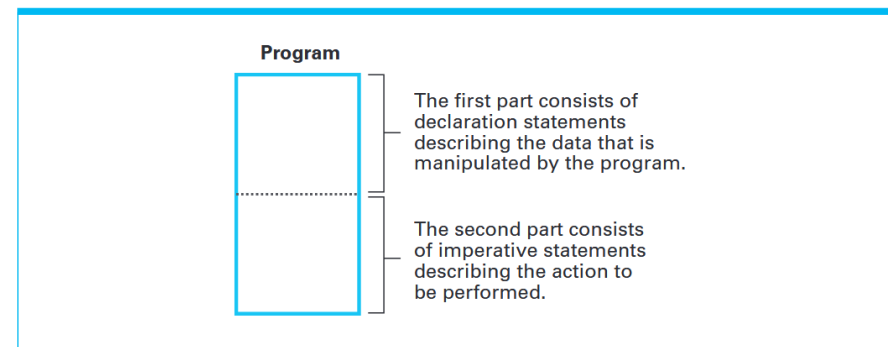
### Program structure

- declarative statements (声明语句)
- imperative statements (命令语句)
- comments (注释)

**Declarative statements** define customized terminology used in a program, such as names for data items.

**imperative statements** describe the steps of the algorithm to be executed.

**Figure 6.4** The composition of a typical imperative program or program unit



**Comments** improve readability by explaining the program to humans.

### Variables and Data Types

**Variable** is a descriptive(描述的) name used to refer to a location in main memory whose value can change during program execution.

```
int WeightLimit;
```

**data type** defines how data is encoded and what operations can be performed on it.

**Common primitive types** include integer, float, character, and Boolean.

**Integer** refers to whole numbers, typically stored using two's complement notation.

```
int WeightLimit = 100;
```

**float(real)** refers to numbers that may contain fractional values and are stored in floating-point notation.

**Boolean** refers to data that can take only the values true or false.

```
if (LimitExceeded) then (...)
```

**Character** represents symbols stored using ASCII or Unicode.

```
char Letter, Digit;
```

### Declaration

In many languages, variables must be declared before they are used.

```
int Height, Width;
```

```
int WeightLimit = 100;
```

A translator can detect errors by checking data types.

### Scripting Languages

**scripting languages(脚本语言)** are a subset of imperative programming languages used to perform administrative or routine tasks.

**Script(脚本)** is a sequence of instructions that describes a task to be performed.

### Data Structure

**data structure** is an organization of data items into a collection that can be accessed and manipulated as a unit.

**Array** is a data structure consisting of a collection of data items of the same type.

```
int Scores[5];
```

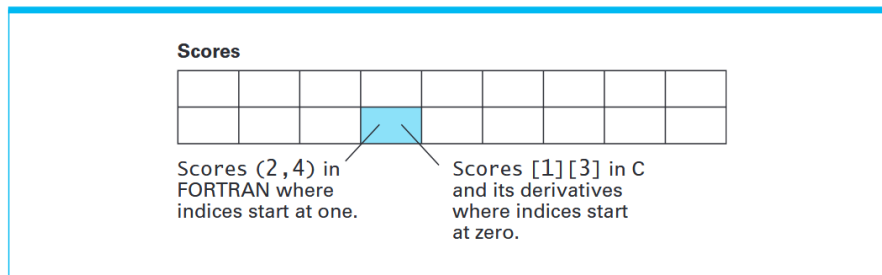
**indices** are used to access individual elements within an array.

```
Scores[0]
```

```
Scores[1]
```

```
array = [10, 20, 30, 40, 50]
```

**Figure 6.5** A two-dimensional array with two rows and nine columns

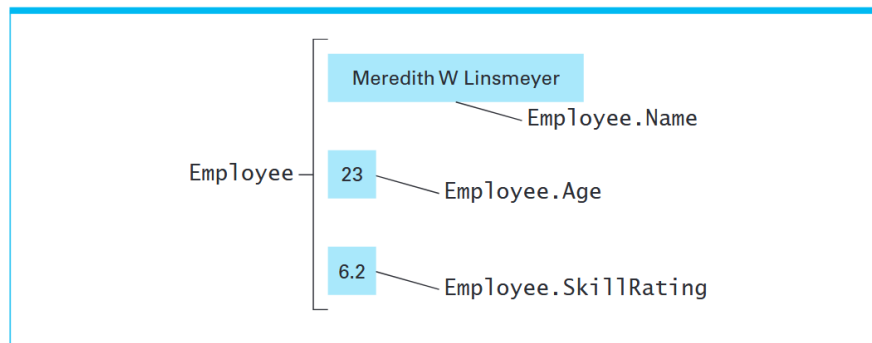


**aggregate type (聚合类型)** is a data structure that groups multiple data items together.

**structure/record/heterogeneous array**

```
struct Student {  
    char name;  
    int age;  
};
```

**Figure 6.6** The conceptual layout of the structure Employee



**Fields** are the individual components of a record.

```
struct Student {  
    char name;    ← field  
    int age;      ← field  
};
```

**Constant** is a data item whose value does not change during program execution.

```
const int Max = 100;
```

### Constants and Literals

**Literal (字面量)** is an explicit value written directly in a program.

```
EffectiveAlt = Altimeter + 645
```

**Text literal (字符串字面量)**

```
LastName = 'Smith'
```

## Constant

A constant is a descriptive name associated with a fixed value.

constant = “有名字的 literal”

## Assignment Statements

**assignment statement** requests that a value be stored in the memory location identified by a variable.

$Z = X + Y$ ; 先算  $X + Y$  把结果存到  $Z$

**operator precedence (运算符优先级)**

$2 * 4 + 6 / 2$

$2 * 4 = 8$

$6 / 2 = 3$

$8 + 3 = 11$

**overloading (运算符重载)** occurs when a symbol has different meanings depending on the data type of its operands.

'abra' + 'cadabra'

abracadabra

## Control Statements

**control statement** alters the execution sequence of a program.

if (KryptoniteLevel < LethalDose):

    RescueDamsel()

else:

    Evade()

**structured programming** is a disciplined approach that uses well-designed control statements to produce readable and reliable programs.

**if-else (分支)**

if (condition) statementA;

else statementB;

**while loop (循环)**

while (condition) { body }

**switch / case (多分支)**

switch (variable) {

    case 'A': statementA; break;

    case 'B': statementB; break;

    case 'C': statementC; break;

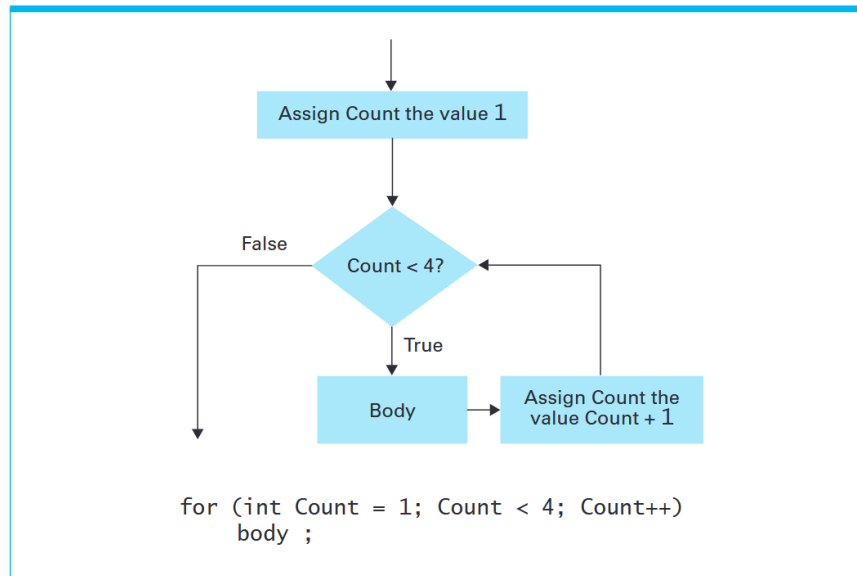
    default: statementD;

}

**for loop (计数循环)**

for (Count = 1; Count <= 3; Count++)

**Figure 6.7** The for loop structure and its representation in C++, C#, and Java



### Comments

**Comments** are explanatory(说明性) statements inserted into a program to help human readers understand the code.

```
/* This is a comment. */
```

```
// This is a comment.
```

### 1. Why is the use of a constant considered better programming style than the use of a literal?

Using constants improves readability and makes programs easier to modify.

### 2. What is the difference between a declarative statement and an imperative statement?

Declarative statements define data items and their properties, whereas imperative statements describe the steps of an algorithm.

declarative → 定义 (变量、类型)

imperative → 执行 (算法步骤)

### 3. List some common data types.

int float char boolean

### 4. Identify some common control structures found in imperative and object-oriented programming languages.

if-else statements, while loops, for loops, and switch (case) statements

### 5. What is the difference between an array and an aggregate type?

An array is an aggregate type whose elements are of the same data type, whereas an aggregate type in general may contain elements of different data types.

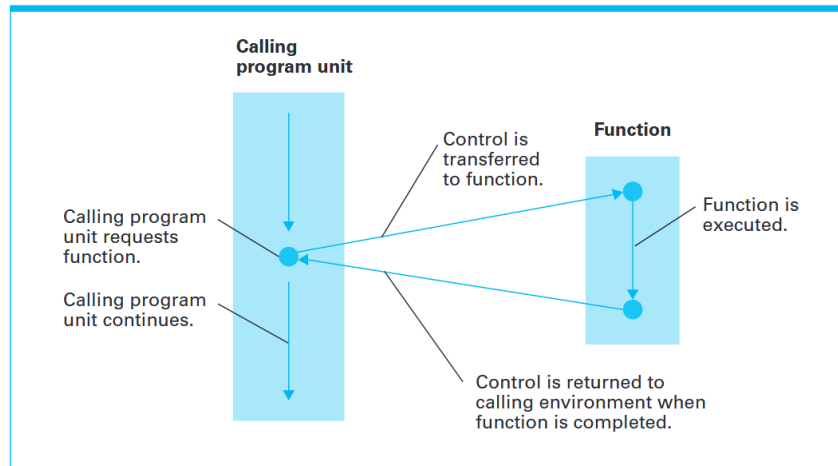


## 6.3 Procedural Units

### Functions

**Function** is a set of instructions for performing a task that can be used by other program units.

**Figure 6.8** The flow of control involving a function



### Function call

The process of transferring control to a function is called calling or invoking the function.

`GetNames()`

`SortNames()`

`WriteNames()`

**Header** is the statement that identifies the function, including its name.

**local variable** is a variable that can be referenced only within the function in which it is declared.

**scope** The scope of a variable is the portion of a program in which the variable can be referenced.

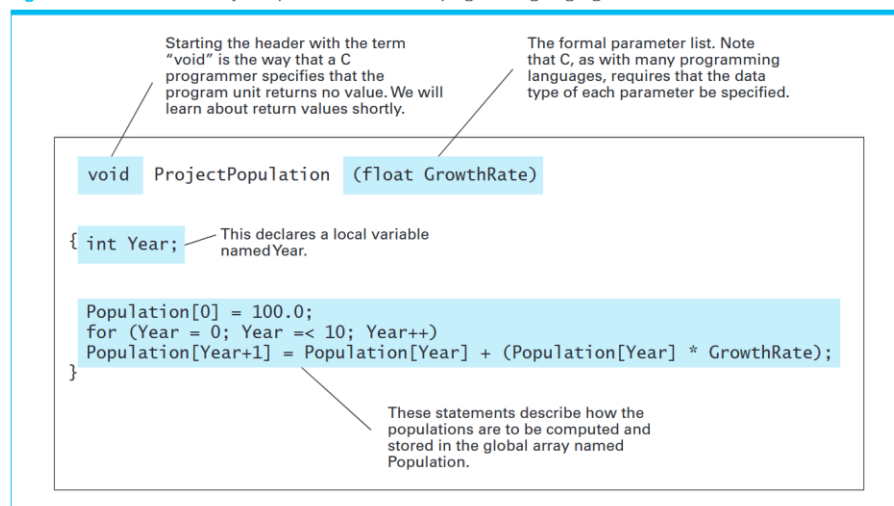
**global variables** are variables whose scope is not restricted to a particular part of a program.

local → 只能函数内用

global → 整个程序都能用

### Parameters

**Figure 6.9** The function `ProjectPopulation` written in the programming language C



**Parameters** are generic terms used in a function that are made specific when the function is applied. 参数是函数中的“通用占位符”，在调用时被具体数据替换。

```
def Sort(List):
```

### formal parameters (形式参数)

Formal parameters are the parameters used within the function definition.

```
def PrintCheck(Payee, Amount): Payee、Amount → formal parameters
```

### actual parameters (实际参数)

Actual parameters are the specific values supplied when the function is called.

```
PrintCheck('John Doe', 150)
```

```
'John Doe' → Payee
```

```
150 → Amount
```

### Matching Rule

Actual parameters are matched with formal parameters in order.

```
PrintCheck('John Doe', 150)
```

### Passed by Value (值传递)

When parameters are passed by value, a copy of the data is given to the function.

值传递是把数据的副本传给函数。函数修改 → 不影响原变量

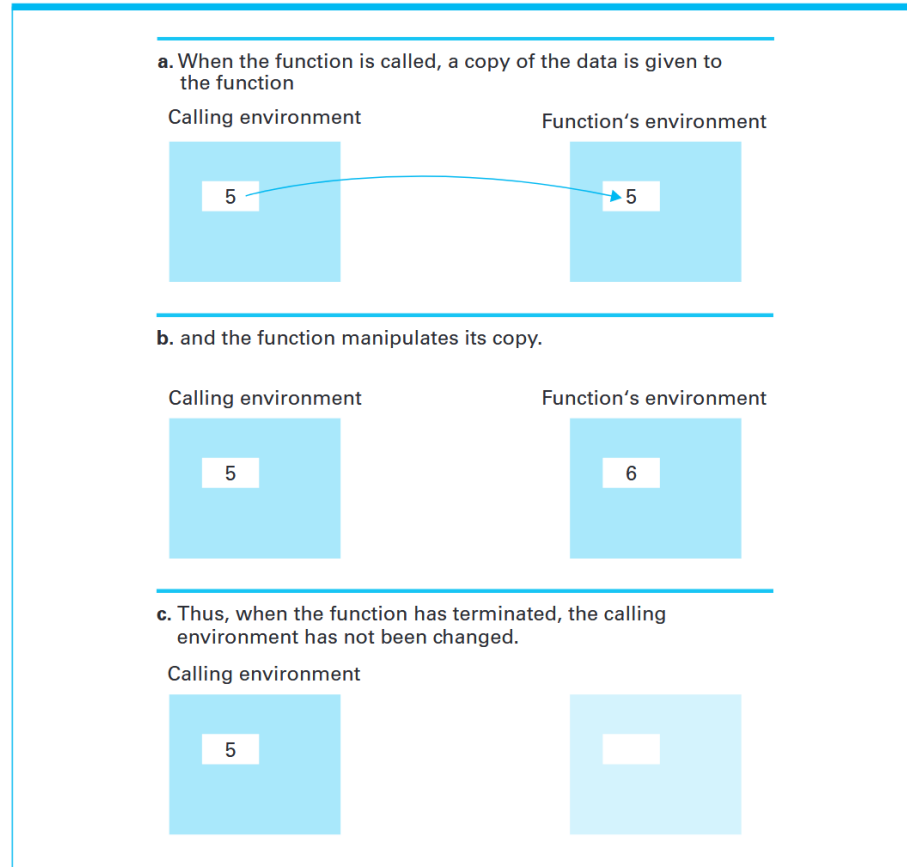
```
def Demo(Formal):
```

```
    Formal = Formal + 1
```

```
Actual = 5
```

```
Demo(Actual)
```

**Figure 6.10** Executing the function Demo and passing parameters by value



### passed by reference (引用传递)

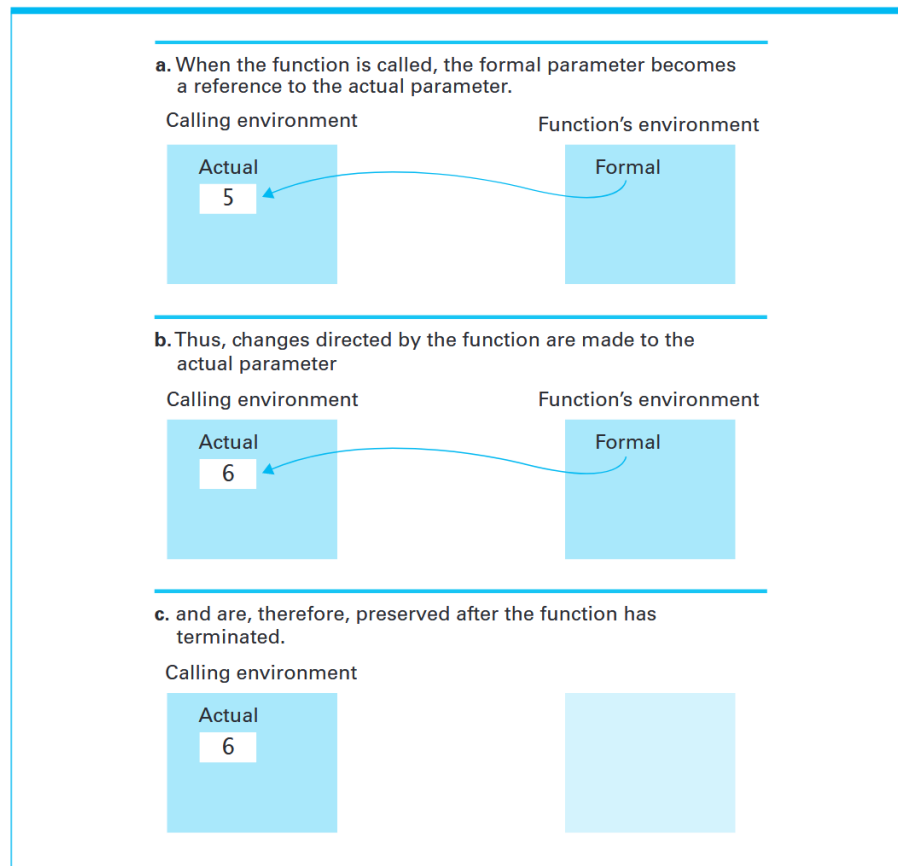
When parameters are passed by reference, the function is given access to the actual data.

函数修改 → 会影响原变量

Actual = 5

Demo(Actual)

**Figure 6.11** Executing the function Demo and passing parameters by reference



### Why both exist

Protects data from unintended modification.

More efficient and allows modification of large data structures.

### Fruitful Functions

A fruitful function is a function that returns a value to the calling program unit.

有返回值的函数

**Procedure** is a program unit that performs an action but does not return a value.

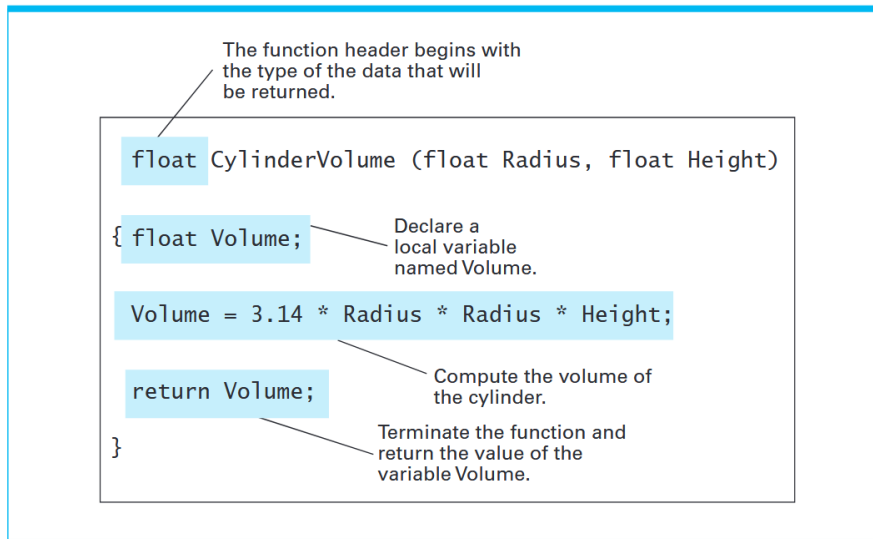
`PrintNames()`

`SortList()`

**Void** is used to indicate that a function returns no value.

`void PrintNames()`

**Figure 6.12** The fruitful function CylinderVolume written in the programming language C



## Event-Driven Software Systems

### event-driven

An event-driven software system is a system in which functions are activated by events rather than explicit function calls. 函数不是被“调用”，而是由“事件触发”。

按钮被点击 → 执行对应函数

**Visual Basic = event-driven + GUI**

**1. What is meant by the “scope” of a variable?**

The scope of a variable is the portion of a program in which the variable can be referenced.

**2. What is the difference between a function and a fruitful function?**

A function may perform an action without returning a value, whereas a fruitful function returns a value to the calling program unit.

**3. Why do many programming languages implement I/O operations as if they were calls to functions?**

Many programming languages implement I/O operations as function calls so that they can be used in a consistent and modular way within programs.

**4. What is the difference between a formal parameter and an actual parameter?**

Formal parameters are the parameters used within the function definition.

Actual parameters are the specific values supplied when the function is called.

**5. What is the difference between a function that passes parameters using call by reference and a function that uses call by value?**

When parameters are passed by value, a copy of the data is given to the function.

When parameters are passed by reference, the function is given access to the actual data.

## 6.4 Language Implementation

### The Translation Process

**Translation** is the process of converting a program from one language to another.

**source program** is the original program written in a high-level language.

**object program** is the translated version in machine-executable form.

source program → translation → object program

$Z = X + Y$

**lexical analyzer (词法分析)** is the process of identifying tokens from the source program.

A **token** is a basic unit such as a number, word, or operator.

lexical analyzer 会忽略 comments

$Z = |X| + |Y|$

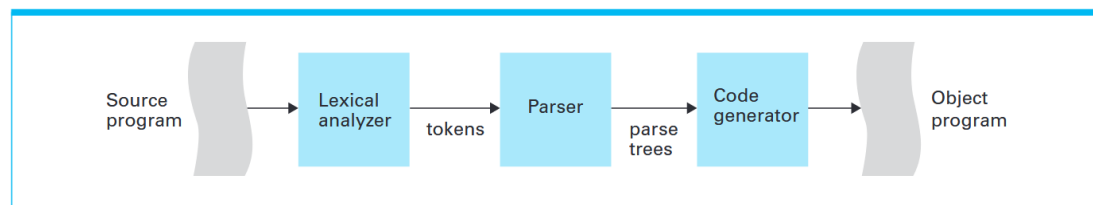
**parser (语法分析)** analyzes the grammatical structure of the token sequence.

检查语法: assignment statement ✓

**code generator (代码生成)** produces the machine-language version of the program.

转换为机器指令 (如 LOAD / ADD / STORE)

Figure 6.13 The translation process



### parser

**fixed-format** require statements to appear in specific positions on the page.

**free-format** do not require specific positioning of statements.

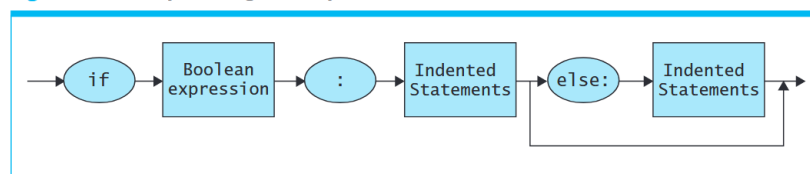
**key words** are words used to mark the structure of a program. ( if, then, else)

**reserved words** are key words that cannot be used for other purposes. `int if = 10;` ×

**grammar** is a set of rules that defines the syntax(语法规则) of a programming language.

**syntax diagrams**

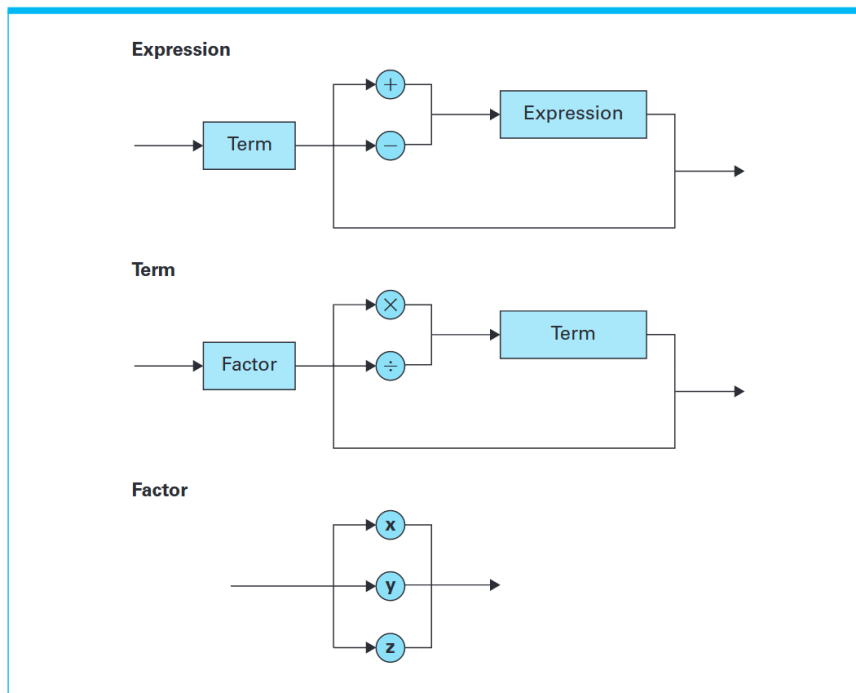
Figure 6.14 A syntax diagram of Python's if-else statement



**nonterminals (非终结符)** are symbols that require further description in the grammar.

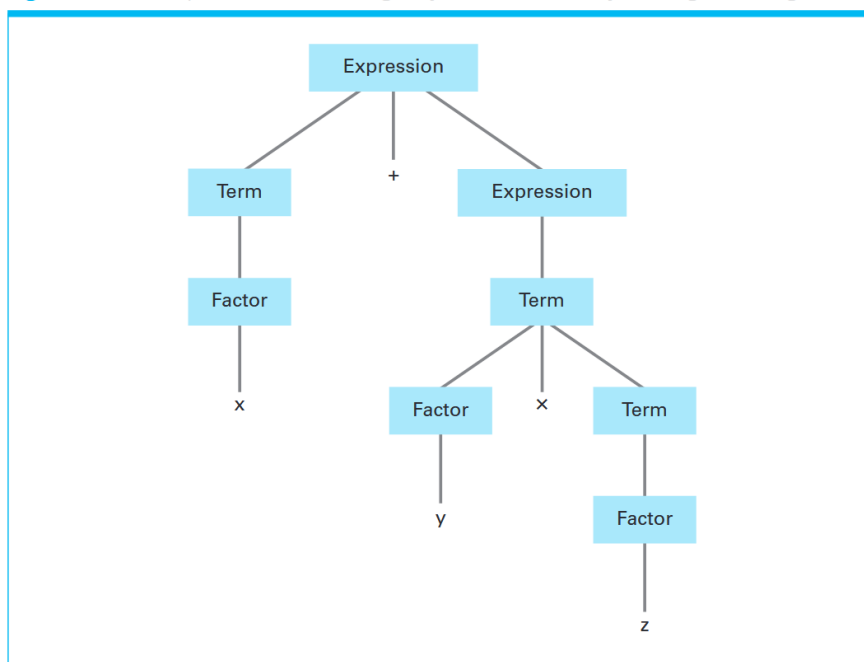
**terminals (终结符)** are the actual symbols or words that appear in the program.

**Figure 6.15** Syntax diagrams describing the structure of a simple algebraic expression



**parse tree** is a pictorial representation of how a string conforms to a grammar.

**Figure 6.16** The parse tree for the string  $x + y \times z$  based on the syntax diagrams in Figure 6.15



**ambiguous grammar(歧义语法)**

if B1:

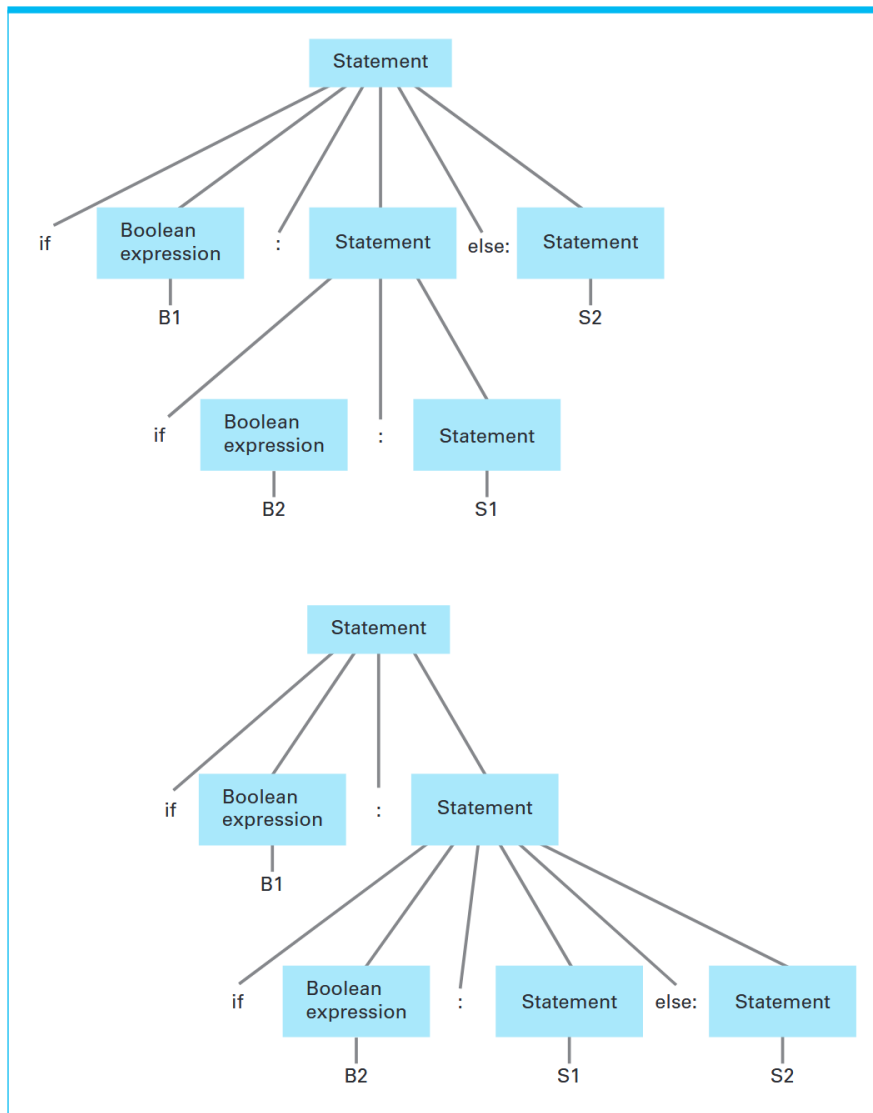
if B2:

S1

else:

S2

**Figure 6.17** Two distinct parse trees for the statement if B1: if B2: S1 else: S2



**symbol table (符号表)** is a table that stores information about identifiers such as variables and their data types.

**-Coercion (类型强制)** is the implicit conversion of data from one type to another.

类型强制 (隐式) 是自动把一种类型转换为另一种。

**-strongly typed (强类型语言)** language requires operations to involve compatible data types.

强类型语言要求所有操作的数据类型必须匹配。

**-type promotion (类型提升)** is a safe form of coercion that converts lower precision data to higher precision.

**-type cast (类型转换)** is an explicit conversion requested by the programmer.

### code generation

**code optimization** is the process of improving the efficiency of the generated machine code.

$x = y + z$

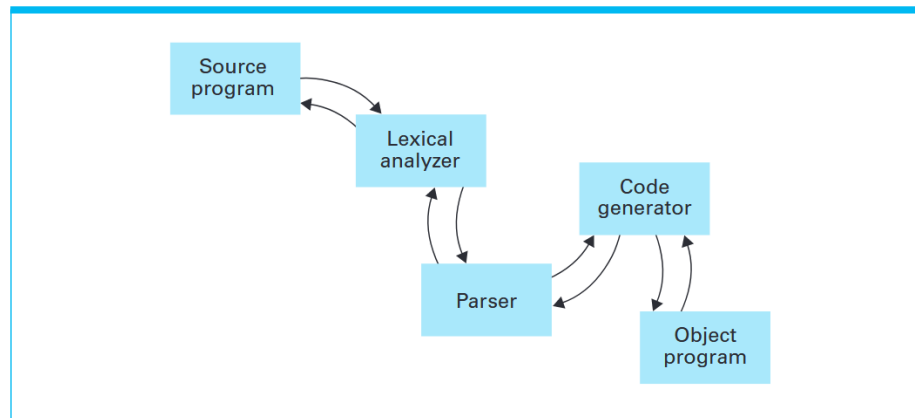
$w = x + z$

However, efficiency can be gained by recognizing that once the first statement has been



executed, the values of x and z will already be in the CPU's general-purpose registers and therefore need not be loaded from memory before performing the second addition.

**Figure 6.18** An object-oriented approach to the translation process



## Implementation of Java and C#

### just-in-time compilation

The intermediate code is translated into machine language just before execution.

JIT 通过在运行时将频繁执行的代码编译为机器码，实现了“跨平台 + 高性能”的结合。

### Software Development Packages

A **software development package** is an integrated collection of software tools used to support the software development process, including editors, translators, and debugging tools.

### Core Components

- **Editor** is used to write and modify source programs.
- **Translator** converts a source program into machine language.
- **Debugging Tools** allow programmers to trace program execution and locate errors.

### Component Architecture (组件化开发)

**Components** are prefabricated(预先做好的模块) software blocks used to build systems.

### 1. Describe the three major steps in the translation process.

Lexical analyze -> parse -> code generate

**Lexical analysis** identifies tokens in the source program.

**Parsing** determines the grammatical structure of the program.

**Code generation** produces the corresponding machine-language instructions.

### 2. What is a symbol table?

A symbol table is a table that stores information about identifiers such as variables and their data types.

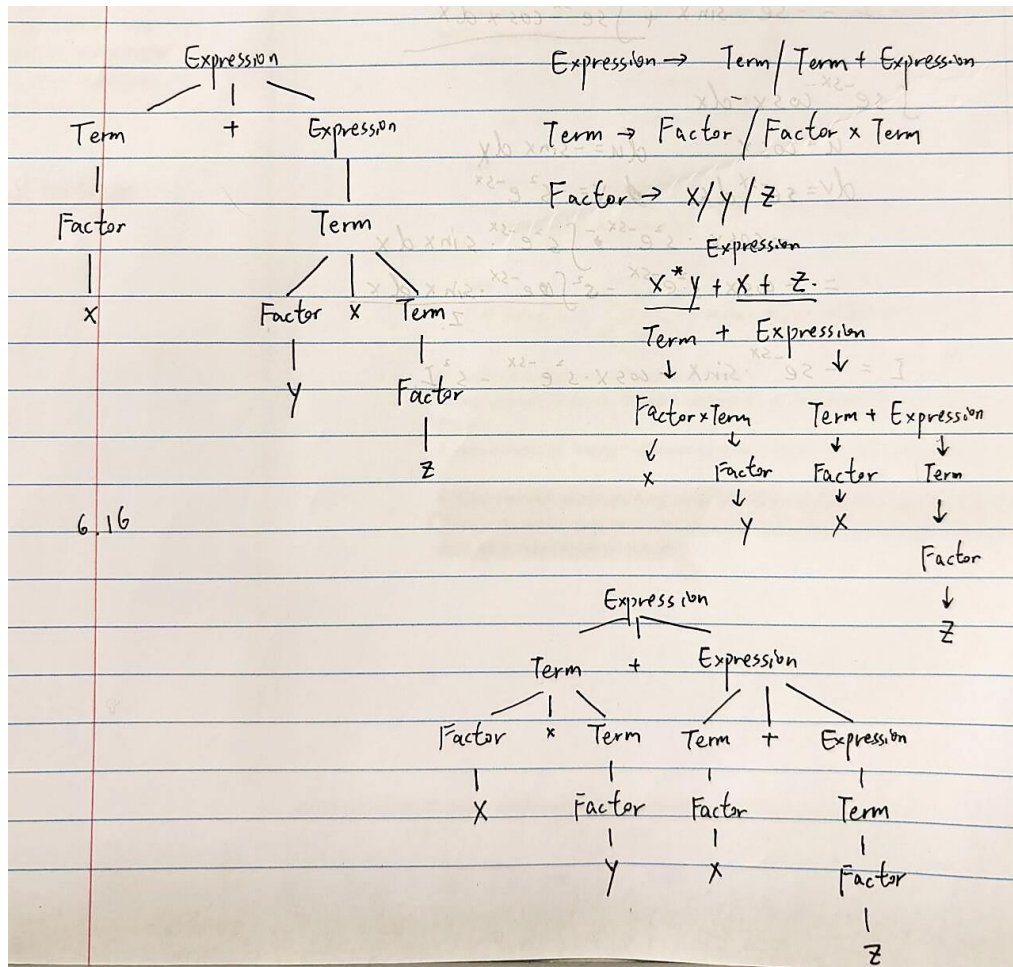
### 3. What is the difference between a terminal and a nonterminal?

A terminal is a symbol that actually appears in the program, whereas a nonterminal represents a structure that must be further defined by the grammar.

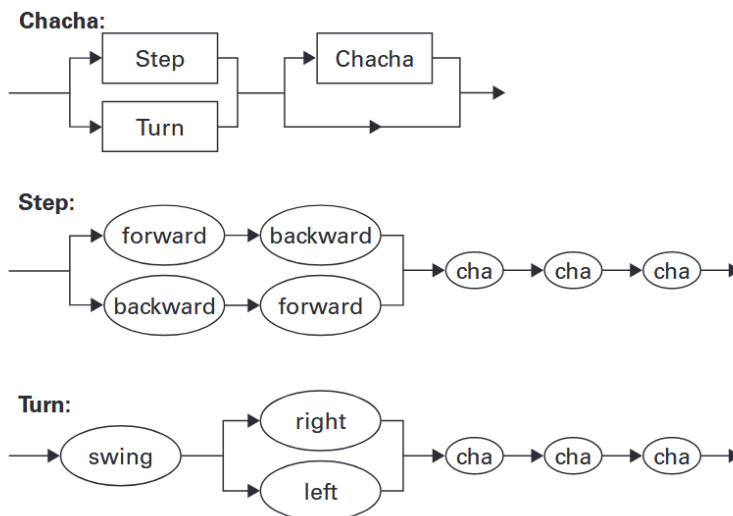
### 4. Draw the parse tree for the expression

$x*y+x+z$

based on the syntax diagrams in Figure 6.15.



5. Describe the strings that conform to the structure Chacha according to the following syntax diagrams.



A Chacha is a sequence of Steps and Turns.

A Step consists of either "forward backward" or "backward forward" followed by "cha cha"

cha".

A Turn consists of "swing" followed by either "right" or "left", followed by "cha cha cha".

**6. Like modern programming language editors, this text uses syntax coloring in an attempt to make code examples easier to read. How many colors have you seen? What does each color seem to mean?**

I have seen several colors used in syntax coloring.

Keywords such as if and while appear in one color,  
identifiers such as variable and function names in another,  
and constants or strings in a different color.

These colors help distinguish different parts of the code and improve readability.

## 6.5 Object-Oriented Programming

**Objects** is an active program unit that contains both data and functions that describe how it responds to stimuli.

对象是一个程序单元：“带数据的功能模块”

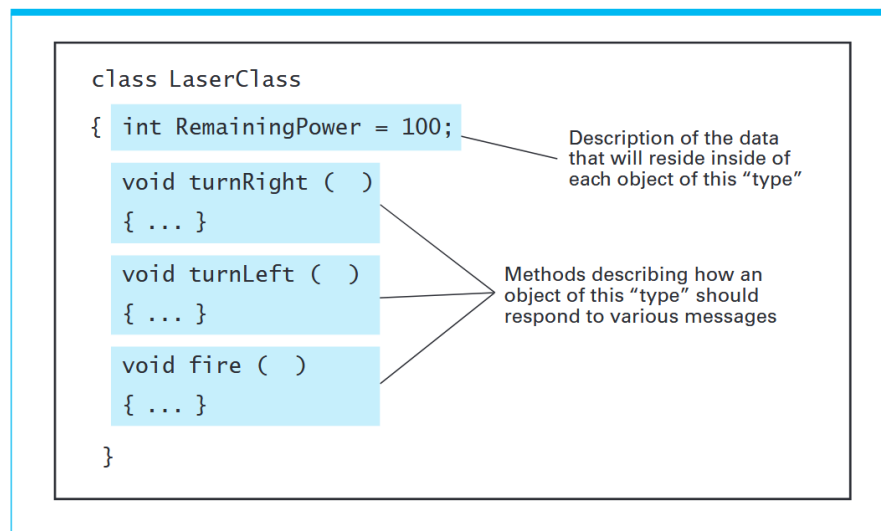
既包含数据（状态）

又包含行为（函数）

### Classes and Objects

A **Class** is a template that describes the common characteristics of a collection of objects.

**Figure 6.19** The structure of a class describing a laser weapon in a computer game



### 概念 类比

class 模具

object 用模具做出来的东西

class LaserClass { ... } 所有 laser 都长这样

LaserClass Laser1, Laser2, Laser3; 创建 3 个具体激光（对象）

An **instance variable**（实例变量） is a variable that resides within an object.

int RemainingPower;

Laser1 有自己的 RemainingPower

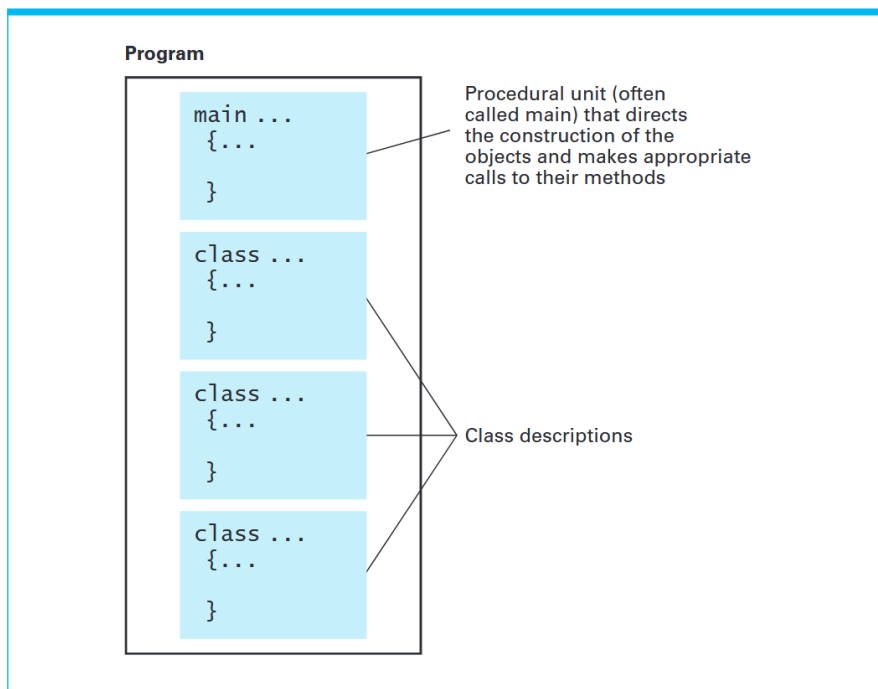
Laser2 有自己的 RemainingPower

A **methods** is a function associated with an object that defines its behavior.

Laser1.fire();

Laser2.turnLeft();

**Figure 6.20** The structure of a typical object-oriented program

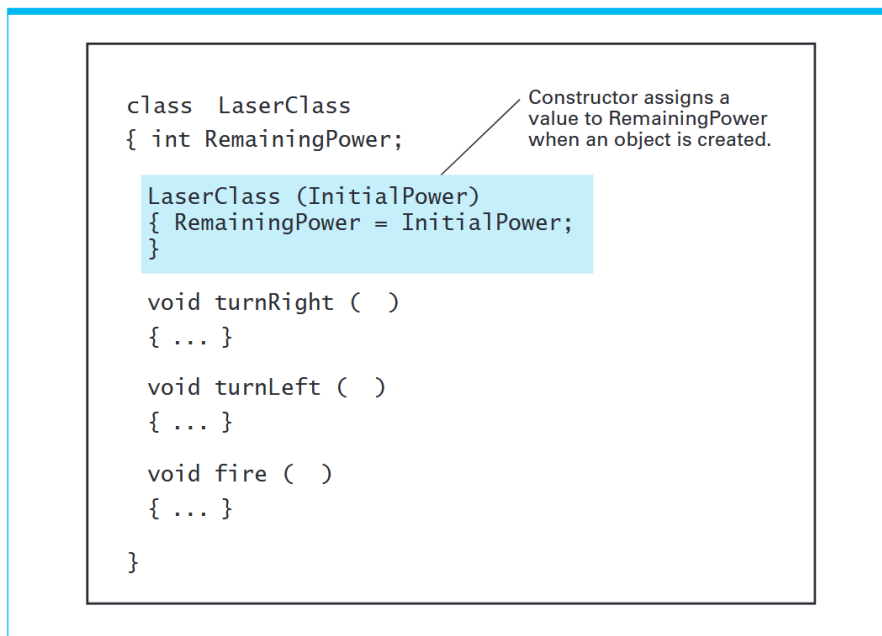


## Constructors 构造函数

A **Constructors** is a special method that is executed automatically when an object is created from a class.

Constructors are used to initialize the instance variables (内部数据) of an object.

**Figure 6.21** A class with a constructor



```
LaserClass Laser1 = new LaserClass(50);
```

```
RemainingPower = 50
```

```
LaserClass Laser2 = new LaserClass(100);
```

如果只使用普通函数，需要额外调用 set 方法

```
class LaserClass {
```

```

    int RemainingPower;

    void setPower(int Power) {
        RemainingPower = Power;
    }
}

LaserClass Laser1;
Laser1.setPower(50);

```

### Additional Features

**Inheritance (继承)** is a mechanism that allows one class to acquire the properties (variables and methods) of another class.

```

class LaserClass { ... }
class RechargeableLaser extends LaserClass {
    void recharge() { ... }
}

```

它自动拥有：

RemainingPower  
turnRight / turnLeft / fire

并且新增：

recharge()

**Polymorphism (多态)** means that the same message (method call) can produce different behaviors depending on the object receiving it.

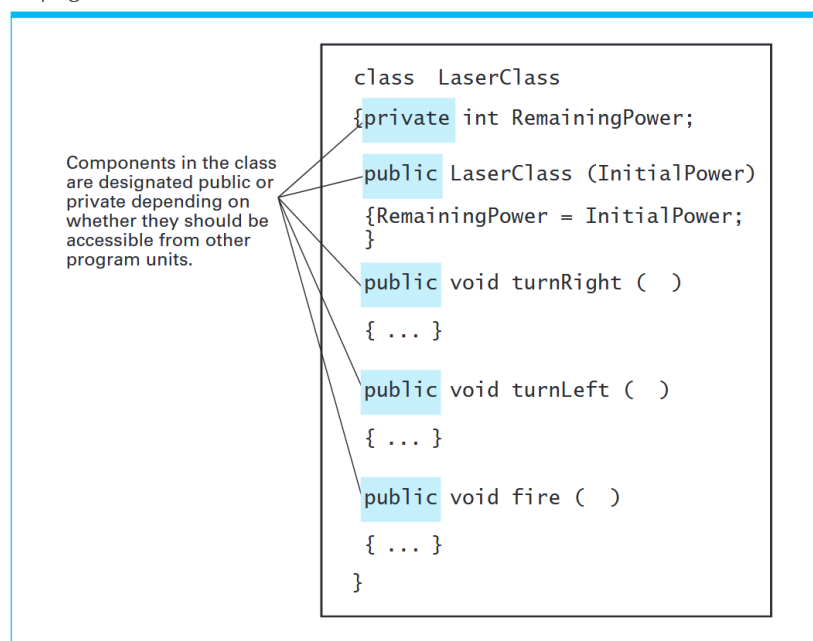
circle (圆) rectangle (矩形) triangle (三角形) 都有 draw()  
shape.draw()不同

**Encapsulation (封装)** is the restriction of access to an object's internal data.

RemainingPower → private ❌ 外部不能直接改

fire() → public ✅ 外部可以调用

**Figure 6.22** Our LaserClass definition using encapsulation as it would appear in a Java or C# program



### 1. What is the difference between an object and a class?

A class is a template that defines the properties and methods of a group of objects, while an object is an instance of that class.

### 2. What classes of objects other than LaserClass might be found in the computer game example used in this section? What instance variables in addition to RemainingPower might be found in the class LaserClass?

Other classes in the game might include:

**Meteor** – representing falling meteors

**Earth** – representing the object to be protected

**ScoreBoard** – tracking the player's score

**Player** – representing the user controlling the lasers

In addition to **RemainingPower**, the class LaserClass might include:

**Position** – location of the laser

**Angle** – direction the laser is aiming

**PowerLevel** – strength of each shot

**Status** – whether the laser is active or not

### 3. Suppose the classes PartTimeEmployee and FullTimeEmployee inherited the properties of the class Employee. What are some features that you might expect to find in each class?

```
class Employee {  
    Name  
    EmployeeID  
  
    function displayInfo()  
}
```

```
class PartTimeEmployee extends Employee {  
    HoursWorked  
    HourlyRate  
  
    function computePay()  
    return HoursWorked * HourlyRate  
}
```

```
class FullTimeEmployee extends Employee {  
    AnnualSalary  
    Bonus  
  
    function computePay()  
    return AnnualSalary + Bonus  
}
```

**4. What is a constructor?**

A Constructors is a special method that is executed automatically when an object is created from a class.

**5. Why are some items within a class designated as private?****Cannot be modified.**

Some items within a class are designated as private to restrict access to them, so that they can only be modified by the object's own methods.



## 6.6 Programming Concurrent Activities

**parallel processing** (or **concurrent processing**) is the simultaneous execution of multiple program activities.

- **parallel processing** Multiple threads execute at the same time on multiple CPUs.
- **concurrent processing** Threads share a single CPU by switching rapidly.

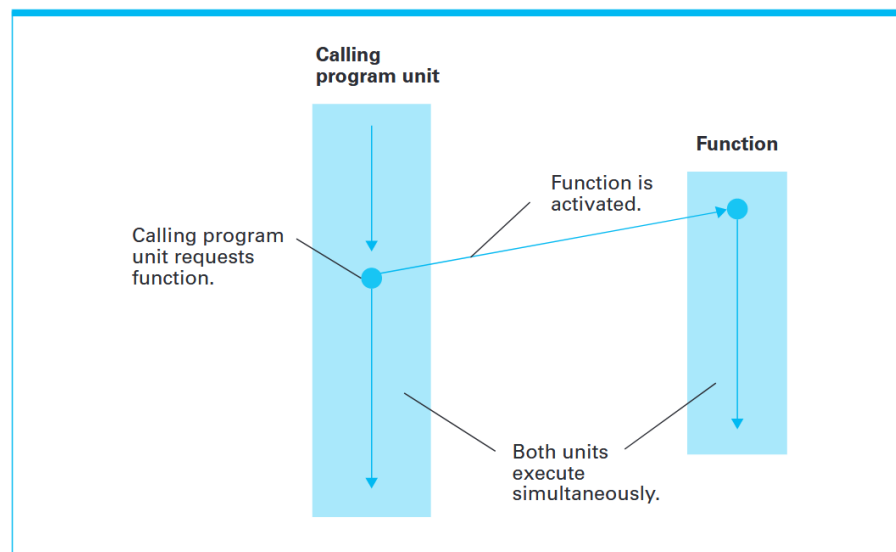
A **thread** is an independent sequence of execution within a program.

### Sequential vs Parallel

传统（函数调用）call function → 等它结束 → 再继续

并发（thread）create thread → 自己继续执行

Figure 6.23 Spawning threads



### Communication & Problems

When multiple threads access the same data, problems can occur if their actions are not coordinated.

假设初始值:  $X = 10$

两个线程 T1 和 T2 同时执行:  $X = X + 3$

步骤 T1            T2

1    读取  $X=10$     读取  $X=10$

2    计算 13        计算 13

3    写回 13        写回 13

最终结果:  $X = 13$  (错)

但正确应该是:  $X = 16$

-> **Race condition**

**Mutual Exclusion (互斥)** Mutually exclusive access means only one thread can access shared data at a time.

**Resolutions:**

- **Threads control access (分散控制)** Each thread is responsible for ensuring exclusive access.

- **Monitor** is a data item that controls access to itself. 数据自带“访问控制机制”（可以理解成“自带锁”）控制权从“线程” → 转移到“数据”

**Programming Smartphones**

Smartphone software is usually developed using general-purpose languages but often requires support for concurrent and event-driven programming.

**1. What are some properties that would be found in a programming language for concurrent processing that would not be found in a more traditional language?**

A programming language for concurrent processing provides mechanisms for creating multiple threads, controlling their execution, and managing communication and synchronization between them, including mutual exclusion.

**2. Describe two methods for ensuring mutually exclusive access to data.**

**Threads control access (分散控制)** Each thread is responsible for ensuring exclusive access.

**Monitor** is a data item that controls access to itself.

**3. Identify some settings other than animation in which parallel computing is beneficial.**

- Weather forecasting
- Air traffic control
- Simulation of complex systems
- Computer networking
- Database maintenance

## 6.7 Declarative Programming

### Logical Deduction (逻辑推理)

Logical deduction is the process of deriving conclusions from known statements using formal rules.

Kermit is on stage OR Kermit is sick

Kermit is not on stage

$\Rightarrow$  Kermit is sick

**inference rules (推理规则)** are formal methods used to derive new statements from existing ones.

**Resolution (归结法)** is an inference rule used to derive a new statement from two clauses.

$P \text{ OR } Q$

$R \text{ OR } \neg Q$

-----

$P \text{ OR } R$

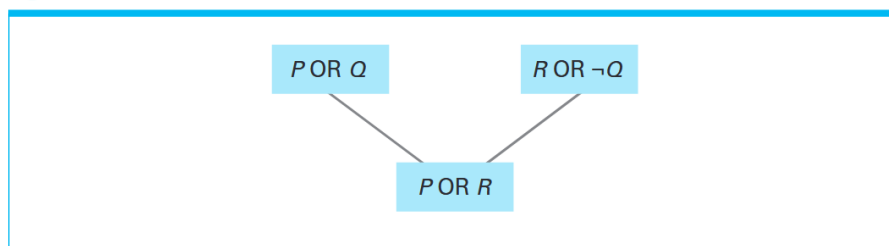
**Resolvent** is the new statement derived from resolution.

$P \text{ OR } R$

### clause form (子句形式)

A clause is a statement in which components are connected by OR.

**Figure 6.24** Resolving the statements  $(P \text{ OR } \neg Q)$  and  $(R \text{ OR } Q)$  to produce  $(P \text{ OR } R)$



### inconsistent (不一致 / 矛盾)

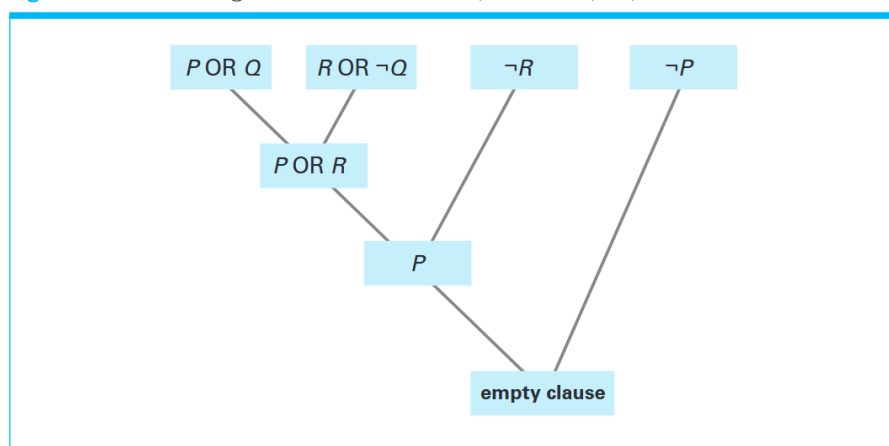
A collection of statements is **inconsistent** if it is impossible for all of them to be true at the same time.

$P$

$\neg P$

If resolution produces the **empty clause**, then the set is inconsistent.

**Figure 6.25** Resolving the statements  $(P \text{ OR } Q)$ ,  $(R \text{ OR } \neg Q)$ ,  $\neg R$ , and  $\neg P$



## 用 Resolution 证明 “implies”

To prove that a set of statements implies P:

Add  $\neg P$  to the set

Apply resolution

If empty clause appears  $\rightarrow$  implication holds

**unification (统一)** is the process of assigning values to variables so that two expressions can match and resolution can be applied.

(Mary is at X)  $\rightarrow$  (Mary's lamb is at X)

Mary is at home

-Step 1: 变 clause form

(Mary's lamb is at X) OR  $\neg$ (Mary is at X)

(Mary is at home)

这两个不能直接 resolve

因为:

Mary is at X

Mary is at home

不完全匹配

-Step 2: Unification

X = home

(Mary's lamb is at home) OR  $\neg$ (Mary is at home)

- Step 3: resolution

(Mary is at home)

resolve  $\rightarrow$

(Mary's lamb is at home)

## Prolog

**logic programming** is a paradigm in which programs consist of logical statements, and computation is performed through deductive reasoning (resolution).

**Prolog** (PROgramming in LOGic) is a logic programming language based on repeated resolution.

**predicates (谓词)** A predicate is a statement that represents a fact or relationship about its arguments.

predicate\_name(argument1, argument2, ...)

谓词 = 表达一个“事实/关系”的基本单位

parent(bill, mary)

Bill is Mary's parent

| 类型            | 规则 |
|---------------|----|
| 常量 (constant) | 小写 |
| 变量 (variable) | 大写 |

**Facts (事实)** A fact is a predicate that is assumed to be true.

faster(turtle, snail). 乌龟比蜗牛快

faster(rabbit, turtle). 兔子比乌龟快

**Rules (规则)** A rule expresses implication.

Conclusion :- Condition.

规则 = “如果……那么……”

写法是反的: 结果 :- 条件

$\text{old}(X) \rightarrow \text{wise}(X)$

$\text{wise}(X) :- \text{old}(X).$

如果 X 老  $\rightarrow$  X 有智慧

$\text{faster}(X, Z) :- \text{faster}(X, Y), \text{faster}(Y, Z).$

如果 X 比 Y 快 且 Y 比 Z 快

$\rightarrow$  X 比 Z 快

**Goals (目标 / 查询)** A goal is a query proposed to the system to be proven.

$\text{faster}(\text{rabbit}, \text{snail}).$

**推理过程**

①  $\text{faster}(\text{rabbit}, \text{snail}).$

**已知 facts (事实)**

$\text{faster}(\text{turtle}, \text{snail}).$

$\text{faster}(\text{rabbit}, \text{turtle}).$

**已知 rule (规则)**

$\text{faster}(X, Z) :- \text{faster}(X, Y), \text{faster}(Y, Z).$

**查询 1: 为什么  $\text{faster}(\text{rabbit}, \text{snail})$  成立?**

$\text{faster}(\text{rabbit}, \text{snail})$

**第一步: 能不能直接匹配 fact? no**

**第二步: 尝试用 rule**

$\text{faster}(X, Z) :- \text{faster}(X, Y), \text{faster}(Y, Z)$

尝试让它匹配

$\text{faster}(\text{rabbit}, \text{snail})$

**变量替换 (unification)**

$X = \text{rabbit}$

$Z = \text{snail}$

$\text{faster}(\text{rabbit}, \text{snail}) :- \text{faster}(\text{rabbit}, Y), \text{faster}(Y, \text{snail})$

**找 Y**

Facts 里  $\text{faster}(\text{rabbit}, \text{turtle})$

$Y = \text{turtle}$

$\text{faster}(\text{rabbit}, \text{turtle}), \text{faster}(\text{turtle}, \text{snail}) \rightarrow \text{faster}(\text{rabbit}, \text{snail})$

②  $\text{faster}(W, \text{snail})$

**Step 1: 先找直接 fact**

$\text{faster}(\text{turtle}, \text{snail})$

$W = \text{turtle}$

**Step 2: 继续找 (Prolog 不会停)**

$\text{faster}(X, Z) :- \text{faster}(X, Y), \text{faster}(Y, Z)$

**匹配目标**

$\text{faster}(W, \text{snail})$

$X = W$

$Z = \text{snail}$

$\text{faster}(W, \text{snail}) :- \text{faster}(W, Y), \text{faster}(Y, \text{snail})$

**找 Y**

faster(turtle, snail)

Y = turtle

faster(W, snail) :- faster(W, turtle), faster(turtle, snail)

再找 W

W = rabbit

1. Which of the statements R, S, T, U, and V are logical consequences of the collection of statements  $(\neg R \text{ OR } T \text{ OR } S)$ ,  $(\neg S \text{ OR } V)$ ,  $(\neg V \text{ OR } R)$ ,  $(U \text{ OR } \neg S)$ ,  $(T \text{ OR } U)$ , and  $(S \text{ OR } V)$ ?

书上方法:

要证明 X 是 logical consequence

把  $\neg X$  加进去如果推出 empty clause (矛盾)

就说明 X 是 logical consequence

R, S, T, U, V  
 $(\neg R \text{ OR } T \text{ OR } S)$ ,  $(\neg S \text{ OR } V)$ ,  $(\neg V \text{ OR } R)$ ,  $(U \text{ OR } \neg S)$ ,  $(T \text{ OR } U)$ ,  $(S \text{ OR } V)$

**[R]**  $\neg R$   $\neg V \text{ OR } R$   $\neg S \text{ OR } V$   $S \text{ OR } V$   $\neg V$

$\neg V$  (circled)  
 $\neg S$   
 $V$   
 $V$   
empty

前提: 所有证词  
你的目标: 证明 S 做了  
反证法: 假设 S 没做 (加入  $\neg S$ )  
推出空: S 没做 会推出 荒谬结论  
消不掉: S 没做 能推出 自治结论 不好说  $\rightarrow$  S 做  
 $\neg R$  错 R 肯定是 logical consequence.

**[S]**  $\neg S$   $S \text{ OR } V$   $\neg V \text{ OR } R$   $\neg R \text{ OR } T \text{ OR } S$   $\neg S$

$V$   
 $R$   
 $T \text{ OR } S$   
 $T$  不是.

**[U]**  $\neg U$   $T \text{ OR } U$

$T$  不是.

**[T]**  $\neg T$   $T \text{ OR } U$

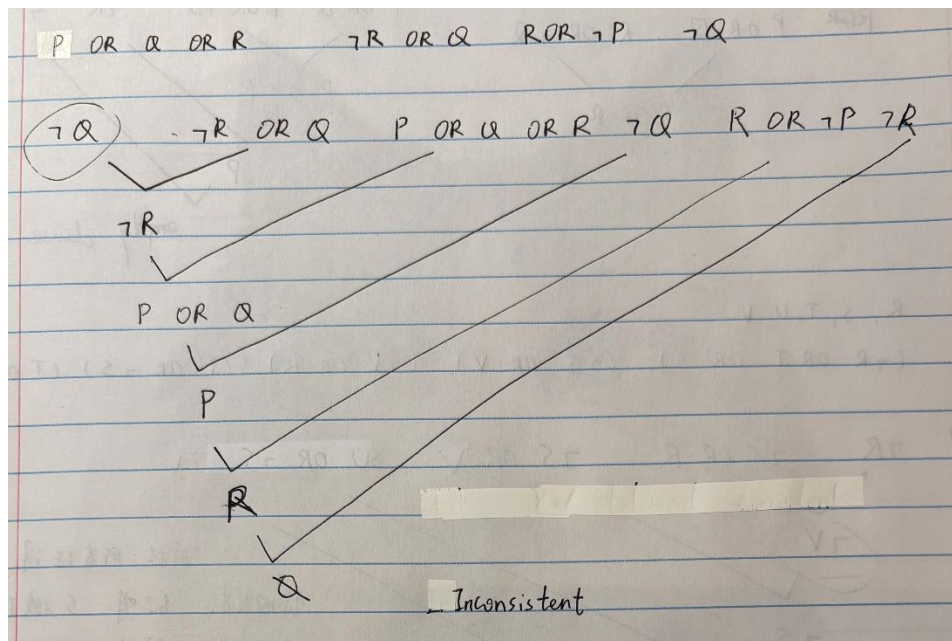
$U$  不是.

**[V]**  $\neg V$   $S \text{ OR } V$   $\neg S \text{ OR } V$   $\neg V$

$S$   
 $V$   
empty  $\neg V$  错 V 是 logical consequence

2. Is the following collection of statements consistent? Explain your answer.

$P \vee Q \vee R$      $\neg R \vee Q$      $R \vee \neg P$      $\neg Q$



3. Complete the two rules at the end of the Prolog program below so that the predicate `mother(X, Y)` means “X is the mother of Y” and the predicate `father(X, Y)` means “X is the father of Y.”

`female(carol).`

`female(sue).`

`male(bill).`

`male(john).`

`parent(john, carol).`

`parent(sue, carol).`

`mother(X,Y) :- parent(X, Y), female(X).`

`father(X,Y) :- parent(X, Y), male(X).`

4. In the context of the Prolog program in question 3, the following rule is intended to mean that X is Y's sibling if X and Y have a common parent.

`sibling(X, Y) :- parent(Z, X), parent(Z, Y).`

What unexpected conclusion would this definition of the sibling relationship allow Prolog to make?

`sibling(X, Y) :-`

`parent(Z, X),`

`parent(Z, Y),`

`X \= Y. % 增加约束: X 不等于 Y`

这个定义会导致 Prolog 认为: 一个人是他自己的兄弟姐妹 (A person is their own sibling)。